

Session 2: Introduction to Linux

1. Working Basics of File System

The Linux file system is organized as a hierarchy starting from the root directory (/). All files and directories originate from this root.

- **File Types in Linux:**
 - **Regular Files:** Text, binary, or data files.
 - **Directories:** Contain other files or directories.
 - **Special Files:** Includes device files and sockets.
 - **File System Hierarchy:**
 - / (Root): Base of the file system.
 - /home: User-specific directories.
 - /bin: Essential binaries like `ls`, `cp`.
 - /etc: Configuration files.
 - /var: Log files and variable data.
-

2. Commands Associated with Files/Directories & Other Basic Commands

- **File and Directory Commands:**
 - `ls`: Lists files and directories.
 - Example: `ls -l` (detailed listing with permissions).
 - `cd`: Changes the directory.
 - Example: `cd /home/user/`.
 - `mkdir`: Creates a new directory.
 - Example: `mkdir new_folder`.
 - `touch`: Creates an empty file.
 - Example: `touch file.txt`.
 - `rm`: Deletes files or directories.
 - Example: `rm file.txt`, `rm -r folder/`.
- **File Content Commands:**
 - `cat`: Displays file content.
 - Example: `cat file.txt`.
 - `head` and `tail`: View the start or end of a file.
 - Example: `head -5 file.txt`, `tail -3 file.txt`.
- **Other Basic Commands:**
 - `pwd`: Displays the current directory.
 - `cp`: Copies files.
 - Example: `cp file1.txt /path/to/destination/`.
 - `mv`: Moves or renames files.

- Example: `mv file.txt newfile.txt`.
-

3. Operators: Redirection and Pipe

- **Redirection:** Redirects input/output between files or commands.
 - `>`: Redirects output to a file (overwrites).
 - Example: `echo "Hello" > file.txt`.
 - `>>`: Appends output to a file.
 - Example: `echo "World" >> file.txt`.
 - `<`: Redirects input from a file.
 - Example: `cat < file.txt`.
 - **Pipe (|):** Connects the output of one command to the input of another.
 - Example: `ls | grep "file"` (lists files containing "file" in the name).
-

4. File Permissions and How to Set Them

Linux uses a permission system to manage file access. Permissions include:

- **Read (r):** Allows viewing the file.
 - **Write (w):** Allows editing the file.
 - **Execute (x):** Allows executing the file as a program.
 - **Viewing Permissions:**
 - Use `ls -l`. Example output:

```
-rw-r--r-- 1 user group 1024 Jan 16 12:00 file.txt
```

 - First character: File type (- for file, d for directory).
 - Next nine characters: Permissions (rw- for owner, r-- for group, r-- for others).
 - **Setting Permissions:**
 - `chmod`: Changes permissions.
 - Example: `chmod 755 file.txt` (owner: read/write/execute; group/others: read/execute).
 - `chown`: Changes ownership.
 - Example: `chown user:group file.txt`.
 - **Access Control Lists (ACLs):**
 - Allow more granular permissions.
 - Example: `setfacl -m u:username:rw file.txt`.
-

5. Network Commands

- **Telnet:** Remote login via command line.
 - Example: `telnet server_address`.
 - **FTP (File Transfer Protocol):** Transfers files between systems.
 - Example: `ftp server_address`.
 - **SSH (Secure Shell):** Secure remote login.
 - Example: `ssh user@server_address`.
 - **SFTP:** Secure File Transfer Protocol.
 - Example: `sftp user@server_address`.
 - **Finger:** Displays user information.
 - Example: `finger username`.
-

6. System Variables

System variables control the behavior of the shell. Common variables include:

- **PS1:** Customizes the command prompt.
 - Example: `export PS1="\u@\h \W]\$ "` (displays username, hostname, and current directory).
 - **PS2:** Secondary prompt for multiline commands.
 - Example: `export PS2="> "`.
 - **Setting Variables:**
 - Use `export` to set them.
 - Example: `export VAR_NAME="value"`.
-

7. Shell Programming

What is Shell?

A **shell** is a command interpreter that allows interaction with the OS. Common shells include:

- **Bash (Bourne Again Shell):** Default shell on most Linux systems.
 - **Zsh (Z Shell):** An enhanced shell with more features.
 - **Ksh (Korn Shell):** Focuses on scripting.
-

8. Shell Variables

Variables store data for use in shell scripts.

- **Types of Variables:**
 - **Local Variables:** Exist within the script or session.
 - Example: `var="hello"`.

- **Environment Variables:** Available globally.
 - Example: `export PATH`.
-

9. Wildcard Symbols

Wildcards match patterns in filenames.

- `*`: Matches any characters.
 - Example: `ls *.txt` (lists all `.txt` files).
 - `?`: Matches a single character.
 - Example: `ls file?.txt`.
 - `[ ]`: Matches a range of characters.
 - Example: `ls file[1-3].txt`.
-

10. Shell Meta Characters

Meta characters have special meanings in the shell.

- `;`: Runs multiple commands sequentially.
 - Example: `echo hello; echo world`.
 - `&&`: Runs the next command only if the previous succeeds.
 - Example: `mkdir new_folder && cd new_folder`.
 - `||`: Runs the next command only if the previous fails.
 - Example: `cd nonexistent || echo "Failed to change directory"`.
-

11. Command Line Arguments

Command-line arguments are inputs passed to scripts or commands.

- Example Script (`script.sh`):

```
echo "First argument: $1"
echo "Second argument: $2"
```

 - Run: `./script.sh arg1 arg2`.
 - Output:

```
First argument: arg1
Second argument: arg2
```
-

12. Read and Echo Commands

- **read**: Reads user input.
 - Example:

```
read name
```

```
echo "Hello, $name!"
```

- **echo:** Outputs text to the terminal.
 - Example: `echo "This is Linux!"`.

This detailed explanation provides a comprehensive understanding of the Linux basics, commands, and shell programming concepts.